

BALANCED-FLOW DEEP FLOW NETWORKS: ZERO-SUM MATRIX PARAMETERIZATION AND DUAL-PRIORITIZED SEARCH FOR DISCRETE OPTIMIZATION SURROGATES

Anonymous authors

Paper under double-blind review

ABSTRACT

Learning neural surrogates for discrete convex-extendible functions is essential for accelerating downstream integer optimization, yet standard Deep Flow Networks (DFNs) suffer from a structural flaw: unconstrained boundary-matrix parameterizations create net-outflow *mass imbalances* across special boundary nodes. These imbalances force excess demand onto artificial high-cost feasibility fallback arcs, corrupting Straight-Through-Estimation (STE) subgradients during training and bottlenecking downstream local descent. To address these pathologies, we introduce **Balanced-Flow Deep Flow Networks (BF-DFN)**, which combine: (1) a *Soft Zero-Sum Matrix Parameterization* that enforces intrinsic boundary mass balance through a column-differencing matrix and a dataset-adaptive regularization strength λ , eliminating demand-dumping distortions; (2) a *Scale-Flexible Column-Rank Core* that maintains minimal row corank and full dynamic range; (3) an *Exact Min-Cost Flow Forward Pass* that preserves discrete M-convex structure; and (4) a *Dual-Prioritized Local Descent Accelerator* that uses optimal dual node potentials μ_S^* to rank elementary candidate swaps by exact marginal cost reductions. Across Convex Quadratic, Minimum-Cost Resource Allocation, and Multiple-Depot Vehicle Scheduling (MDVSP) benchmarks, BF-DFN achieves up to a **2.17 $\times$ lower normalized test MSE** (1.322×10^{-3} vs. 2.873×10^{-3} on Quadratic) and accelerates downstream integer optimization search by **22 $\times$ –79 $\times$** in solver evaluations, while eliminating STE subgradient variance spikes and matching standard DFN on flow-native tasks via its adaptive balance knob.

1 INTRODUCTION

Predict-and-optimize pipelines and neural surrogate modeling for discrete optimization have emerged as crucial paradigms for bridging statistical learning with downstream decision-making (Wilder et al., 2018; Ferber et al., 2019; Kool et al., 2018). In operational domains such as resource allocation, vehicle scheduling, and network-flow routing, objective functions frequently display discrete *convex-extendible* structure (Moriguchi et al., 2017; Ahuja et al., 1993). Embedding constrained optimization primitives directly into deep architectures through differentiable optimization layers (Amos & Kolter, 2017; Agrawal et al., 2019; Ma et al., 2026) enables end-to-end learning of decision-focused representations. However, modeling discrete convex-extendible functions over integer lattices poses a fundamental challenge: neural surrogates must accurately capture complex nonlinear objective surfaces while remaining computationally tractable for downstream inference-time integer optimization.

Recently, Deep Flow Networks (DFNs) were proposed as structural neural surrogates that model convex-extendible functions over integer domains via exact minimum-cost flow (MCF) problems on directed graphs (Ahuja et al., 1993; Goldberg & Tarjan, 1990). Given an integer input $x \in \mathbb{Z}^d$, a DFN maps x through an integer boundary matrix $A \in \mathbb{Z}^{p \times d}$ and bias $b \in \mathbb{Z}^p$ into net outflows $a_i^\top x + b_i$ across p special boundary nodes $S = \{v_1, \dots, v_p\}$. While standard DFNs possess theoretical universality for convex-extendible functions, they suffer from a severe architectural pathology: unconstrained parameterization of A and b introduces arbitrary total net-outflow imbalances

$\sum_{i=1}^p (a_i^\top x + b_i) \neq 0$. To guarantee feasibility during training, standard DFNs dump these imbalances onto artificial high-cost feasibility fallback arcs or extra balancing nodes.

This demand-dumping mechanism degrades both model optimization and downstream local search. During backpropagation, the integer parameters A , b , and edge capacities u are updated using Straight-Through Estimation (STE) (Bengio et al., 2013; Courbariaux et al., 2015; Ajanthan et al., 2019). When boundary net outflows are unbalanced, large penalty costs dominate the dual node potentials, corrupting STE subgradients (Yin et al., 2019) and yielding high training variance on structured flow tasks. Furthermore, at inference time, minimizing the learned DFN objective via standard local search scans all $p(p-1)$ candidate elementary swaps $e_i - e_j$ at each iterate, incurring $O(p^2)$ query complexity per step and a runtime bottleneck.

To overcome these structural and computational limitations, we propose **Balanced-Flow Deep Flow Networks (BF-DFN)**. BF-DFN introduces a dataset-adaptive **Soft Zero-Sum Matrix Parameterization** $A = C \text{round}(V \text{diag}(s)) + \text{round}(R)$ and $b = C \text{round}(\bar{v}) + \text{round}(r_b)$, where $C \in \{-1, 0, 1\}^{p \times (p-1)}$ is a fixed zero-sum column-differencing matrix ($\mathbf{1}_p^\top C = \mathbf{0}^\top$) applied *after* rounding the integer core to guarantee exact mass balance. A soft penalty $\lambda(\|R\|_F^2 + \|r_b\|_2^2)$ serves as a flexible balance knob: for convex targets (e.g., Quadratic, Resource Allocation), $\lambda > 0$ confines A to the zero-sum subspace $\mathbf{1}_p^\top A = \mathbf{0}^\top$, guaranteeing intrinsic mass balance $\sum_{i=1}^p (a_i^\top x + b_i) \equiv 0$, removing reliance on penalty arcs and eliminating STE corruption; for flow-native structures (e.g., MDVSP), $\lambda = 0$ smoothly recovers full DFN expressivity without redundant parameter dynamics. To prevent dynamic-range collapse and fix the row corank $k = p - d$ at its minimum, the core uses a scale-flexible matrix $V \text{diag}(s)$ with positive learnable factors $s_j = 1 + \text{softplus}(\bar{s}_j)$. Finally, for downstream local descent, our **Dual-Prioritized Local Descent** accelerator extracts exact dual node potentials μ_S^* from a single MCF solve at iterate x and ranks candidate swaps $e_i - e_j$ in descending order of $\mu_j^* - \mu_i^*$ (the exact first-order marginal cost reduction), reaching minimizers orders of magnitude faster.

We evaluate BF-DFN on Convex Quadratic, Minimum-Cost Resource Allocation (RA), and Multiple-Depot Vehicle Scheduling (MDVSP) against standard DFN, an MLP, a LogSumExp-Transform (LSET) surrogate, and an Input Convex Neural Network (ICNN) (Amos et al., 2016). BF-DFN attains up to $2.17\times$ lower normalized test MSE while matching DFN on flow-native MDVSP, and Dual-Prioritized Local Descent yields $22\times$ – $79\times$ speedups in solver calls at equal or better solution quality. Our contributions are:

- **Soft Zero-Sum Matrix Parameterization** with a dataset-adaptive penalty knob λ that enforces intrinsic flow mass balance, eliminating demand-dumping distortions and STE subgradient corruption (Sec. 3.2).
- **Scale-Flexible Column-Rank Conditioning** via a full-column-rank zero-sum core $V \text{diag}(s)$ with learnable per-column scales, ensuring full dynamic range and minimal row corank (Sec. 3.3).
- **Dual-Prioritized Local Descent**, a sensitivity-guided search that ranks elementary swaps by optimal dual potentials, cutting candidate evaluations by up to $79\times$ (Sec. 3.5).
- **Comprehensive empirical and ablation validation**: main benchmarks plus five targeted ablations dissecting soft vs. hard balance, candidate ordering, core conditioning, neighborhood truncation, and penalty-arc sensitivity (Sec. 4).

2 RELATED WORK

Neural surrogates for discrete and integer optimization. Surrogate modeling replaces expensive combinatorial solvers with trainable neural architectures (Dumouchelle et al., 2022; Ferber et al., 2022). Unconstrained surrogates such as MLPs and LSET models fit objective surfaces without structural priors, but inference-time optimization over them requires embedding the network into mixed-integer programs via big-M formulations (Lu & Kumar, 2019) or disjunctive partitions (Tsay et al., 2021), adding integer variables per activation and causing exponential solve-time scaling (Manchanda & Ranu, 2022; Tsay, 2026). Structural surrogates instead embed mathematical properties into the architecture: Input Convex Neural Networks (Amos et al., 2016) enforce convexity for efficient continuous inference (Liu & Oliveira, 2025; Liu & Hu, 2026), and differentiable

Frank-Wolfe layers (Liu et al., 2023) handle constrained continuous domains. Yet continuous convex surrogates lack combinatorial priors (submodularity, M-convexity) and cannot model discrete step interactions on integer lattices. BF-DFN parameterizes discrete extendible functions through network flows, yielding structured local-search landscapes that avoid big-M encodings.

Deep flow networks and discrete M-convexity. Network-flow function approximators leverage MCF formulations (Ahuja et al., 1993; Goldberg & Tarjan, 1990) to model discrete convex functions. In discrete convex analysis, functions satisfying discrete midpoint convexity (Moriguchi et al., 2017) and M-convex exchange properties (Sakaue & Oki, 2022; Oki & Sakaue, 2023) are central for polynomial-time minimizability. DFNs exploit this by parameterizing edge costs, capacities, and boundary matrices A to fit convex-extendible functions; differentiable MCF layers have also aided multi-object tracking and vehicle scheduling (Schulter et al., 2017; Zhao & xiang Wu, 2024). Despite their expressivity, standard DFNs incur outflow imbalances when A is unconstrained: excess supply is dumped onto high-cost penalty arcs, generating extreme dual-potential spikes that corrupt STE subgradients (Bengio et al., 2013; Yin et al., 2019). BF-DFN resolves this via a soft zero-sum parameterization that enforces intrinsic balance when $\lambda > 0$ while retaining full DFN expressivity when $\lambda = 0$.

Local-search acceleration in combinatorial optimization. Local search is ubiquitous in discrete optimization, but its query complexity limits large-scale efficiency (Brânzei & Li, 2020). For M-convex minimization, steepest descent explores an elementary neighborhood of unit swaps $x + e_i - e_j$ (Sakaue & Oki, 2022); naively evaluating all $p(p - 1)$ swaps requires $O(p^2)$ objective evaluations per step (Ambrus & Heck, 2025). Sensitivity-guided and dual-variable methods use marginal price signals to direct exploration (Bertsekas, 1988; Korotin et al., 2022), and recent work integrates learned predictions or dual bounds to warm-start relaxations (Chen et al., 2026; Oki & Sakaue, 2023). BF-DFN extracts optimal dual node potentials μ_S^* from exact MCF solves and ranks swaps by $\mu_j^* - \mu_i^*$, the exact first-order marginal reduction, cutting evaluations by up to $79\times$ without compromising solution quality.

3 METHODOLOGY

3.1 PROBLEM FORMULATION AND DEEP FLOW NETWORK PRELIMINARIES

A Deep Flow Network is a tuple $\Pi = (G, c, u, S, A, b)$, where $G = (V, E)$ is a directed graph with $n = |V|$ nodes and $m = |E|$ edges; $c \in \mathbb{Q}^m$ are edge costs; $u \in \mathbb{Z}_{\geq 0}^m$ are integer capacities; $S = \{v_1, \dots, v_p\} \subseteq V$ are p boundary nodes; and $A \in \mathbb{Z}^{p \times d}$, $b \in \mathbb{Z}^p$ are integer boundary transformations. Given $x \in \mathbb{Z}^d$, each $v_i \in S$ is assigned an input-dependent net outflow $a_i^\top x + b_i$, while internal nodes $v \in V \setminus S$ have zero net outflow. The induced value $f(x; \Pi)$ solves the exact minimum-cost integer flow problem

$$\begin{aligned} f(x; \Pi) = \min_{\xi \in \mathbb{Z}^m} \sum_{e \in E} c_e \xi_e \quad \text{s.t.} \quad & \sum_{e \in \delta^+(v_i)} \xi_e - \sum_{e \in \delta^-(v_i)} \xi_e = a_i^\top x + b_i, \quad \forall v_i \in S, \\ & \sum_{e \in \delta^+(v)} \xi_e - \sum_{e \in \delta^-(v)} \xi_e = 0, \quad \forall v \in V \setminus S, \quad 0 \leq \xi_e \leq u_e, \quad \forall e \in E, \end{aligned} \quad (1)$$

where $\delta^+(v)$ and $\delta^-(v)$ are the outgoing and incoming edges at v ; if equation 1 is infeasible, $f(x; \Pi) = +\infty$. A necessary feasibility condition in any directed network is mass conservation: the boundary net outflows must sum to zero, $\sum_{i=1}^p (a_i^\top x + b_i) = 0$. In standard DFNs the rows of A and b are parameterized independently, so $\mathbf{1}_p^\top A \neq \mathbf{0}_d^\top$ and $\mathbf{1}_p^\top b \neq 0$, and total net outflow varies with x . To avoid infeasibility, standard DFNs rely on artificial balancing nodes or high-cost bipartite fallback arcs with penalty cost $c_{\text{penalty}} \gg 0$. As illustrated in Figure 1, dumping demand imbalances onto these arcs induces severe gradient distortions during training.

3.2 SOFT ZERO-SUM MATRIX PARAMETERIZATION

To eliminate imbalance, we anchor the parameterization in the zero-sum subspace using a fixed column-differencing matrix $C \in \{-1, 0, 1\}^{p \times (p-1)}$ with $C_{i,j} = 1$ if $i = j$, $C_{i,j} = -1$ if $i = j + 1$, and 0 otherwise. Every column of C sums to zero ($\mathbf{1}_p^\top C = \mathbf{0}_{p-1}^\top$). We then form the boundary

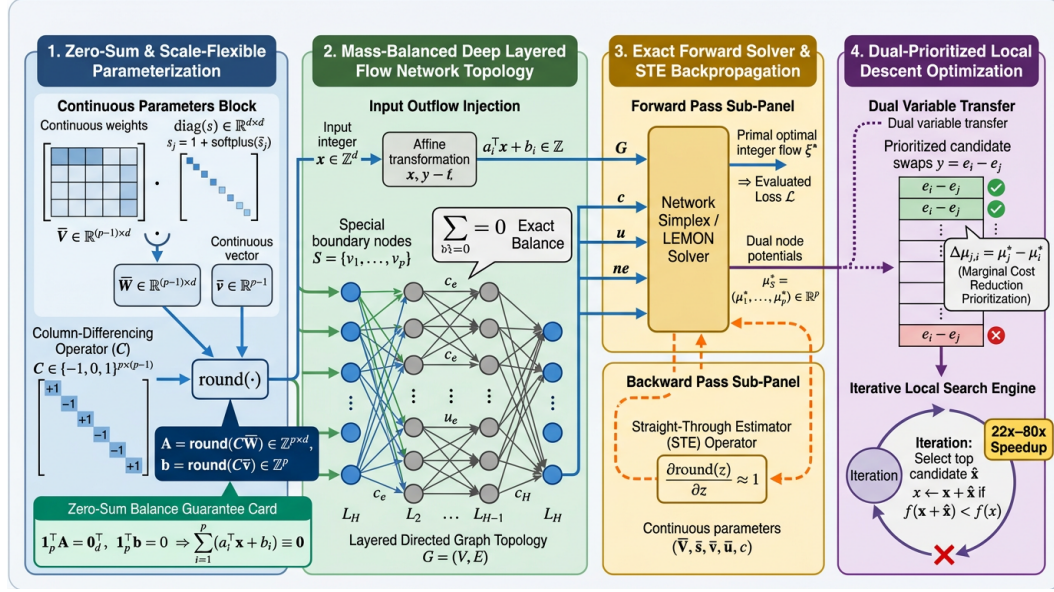


Figure 1: End-to-end architecture of Balanced-Flow Deep Flow Networks (BF-DFN). (1) Continuous weights $\bar{V}$ and diagonal scales $\text{diag}(s)$ are rounded and combined with a zero-sum column-differencing operator C to produce integer boundary parameters A, b with exact mass conservation $\mathbf{1}_p^\top A = \mathbf{0}_d^\top$. (2) Input $x \in \mathbb{Z}^d$ yields net outflows $a_i^\top x + b_i$ at boundary nodes S of a layered graph G . (3) The forward pass solves an exact integer min-cost flow (Network Simplex/LEMON) to evaluate $\mathcal{L}$ and extract dual potentials μ_S^* , with STE for backpropagation. (4) Downstream, μ_S^* prioritizes swaps $e_i - e_j$ by marginal reduction $\mu_j^* - \mu_i^*$, accelerating local descent by $22\times\text{--}79\times$.

matrix and bias as

$$A = C \text{round}(\bar{W}) + \text{round}(R), \quad b = C \text{round}(\bar{v}) + \text{round}(r_b), \quad (2)$$

where $\bar{W} \in \mathbb{R}^{(p-1) \times d}$ and $\bar{v} \in \mathbb{R}^{p-1}$ define the zero-sum core, and $R \in \mathbb{R}^{p \times d}$, $r_b \in \mathbb{R}^p$ are learnable residuals. The order of operations is essential for *exact* integer conservation: because element-wise rounding is non-linear, $\text{round}(C\bar{W}) \neq C \text{round}(\bar{W})$ in general, so rounding *after* column differencing would break the zero-sum structure on the integer lattice, whereas $C \text{round}(\bar{W})$ keeps A in the exact integer column space of C , guaranteeing $\mathbf{1}_p^\top A = \mathbf{0}_d^\top$ for any integer core (Appendix A). During training we add a soft balance penalty to the standard MSE loss,

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{MSE}} + \lambda (\|R\|_F^2 + \|r_b\|_2^2), \quad \lambda \geq 0. \quad (3)$$

The strength λ acts as a continuous structural knob. When $\lambda > 0$, the penalty drives $R \rightarrow \mathbf{0}$ and $r_b \rightarrow \mathbf{0}$, forcing A into the exact zero-sum subspace $\mathbf{1}_p^\top A = \mathbf{0}_d^\top$, $\mathbf{1}_p^\top b = 0$. This guarantees $\sum_{i=1}^p (a_i^\top x + b_i) \equiv 0$ for all $x \in \mathbb{Z}^d$ (Appendix A), rendering all flow instances intrinsically mass-balanced and eliminating routing over high-cost feasibility arcs. Conversely, for flow-native structures such as MDVSP, $\lambda = 0$ relaxes the restriction to a single free matrix (standard DFN), enabling full expressivity without redundant parameter dynamics.

3.3 SCALE-FLEXIBLE FULL COLUMN-RANK CONDITIONING

While zero-sum balance eliminates infeasibility, the core $\bar{W}$ must retain sufficient rank and dynamic range to match target scales without ill-conditioning. The complexity parameter $\Delta(A) = 2 \cdot (2k)^{2k} \det^*(A)$ governing local descent grows with the row corank $k = p - \text{rank}(A)$; minimizing k requires $\text{rank}(A) = d$. We structure the core as

$$\bar{W} = \bar{V} \cdot \text{diag}(s_1, \dots, s_d), \quad s_j = 1 + \text{softplus}(\bar{s}_j) > 0, \quad (4)$$

where $\bar{V} \in \mathbb{R}^{(p-1) \times d}$ is initialized with linearly independent columns ($\text{rank}(\bar{V}) = d$) and $\bar{s}_j$ enable per-column scaling with $s_j \geq 1$. This fixes the corank at its minimum $k = p - d$ while granting A full dynamic-range flexibility, preventing range collapse.

Algorithm 1 Dual-Prioritized Local Descent

Require: Feasible start $x \in \mathcal{X}$, DFN $\Pi = (G, c, u, S, A, b)$, truncation ratio $K \in (0, 1]$.

```

1: loop
2:   Evaluate  $\hat{f}(x; \Pi)$  via MCF solver and extract optimal dual potentials  $\mu_S^* = (\mu_1^*, \dots, \mu_p^*)^\top$ .
3:   Form candidate swaps  $\mathcal{P} = \{(i, j) \in [p] \times [p] : i \neq j\}$ ; sort by descending  $\mu_j^* - \mu_i^*$ .
4:   Truncate to the top  $\lceil K \cdot |\mathcal{P}| \rceil$  pairs, yielding  $\mathcal{P}_{\text{top}}$ ; set improved  $\leftarrow$  false.
5:   for all  $(i, j) \in \mathcal{P}_{\text{top}}$  do
6:     Find integer  $\hat{x} \in \mathbb{Z}^d$  with  $A\hat{x} = e_i - e_j$  and  $x + \hat{x} \in \mathcal{X}$ .
7:     if  $\hat{x}$  exists and  $\hat{f}(x + \hat{x}; \Pi) < \hat{f}(x; \Pi)$  then
8:        $x \leftarrow x + \hat{x}$ ; improved  $\leftarrow$  true; break
9:     end if
10:  end for
11:  if not improved then
12:    return local minimizer  $x$ 
13:  end if
14: end loop

```

3.4 EXACT FORWARD PASS AND STRAIGHT-THROUGH ESTIMATION

For every $x \in \mathbb{Z}^d$, the forward pass solves the exact integer MCF equation 1 via the Network Simplex solver in LEMON (Ahuja et al., 1993), preserving discrete M-convex values and exact conservation. Gradients propagate to $(\bar{V}, \bar{s}, \bar{v}, \bar{u}, c)$ via STE (Bengio et al., 2013; Schuler et al., 2017); nonnegative capacities use $u = \text{round}(\text{softplus}(\bar{u}))$, and the rounding operator is treated as identity in the backward pass, $\partial \text{round}(z)/\partial z \approx 1$. In standard DFNs, imbalance flow over fallback arcs with $c_{\text{penalty}} \gg 0$ inflates optimal dual potentials μ_v^* to $O(c_{\text{penalty}})$; STE subgradients w.r.t. A then receive noise proportional to $c_{\text{penalty}} \cdot \partial \mathcal{L}/\partial f$, causing extreme variance spikes. In BF-DFN, $\lambda > 0$ enforces $\mathbf{1}_p^\top A = \mathbf{0}_d^\top$, ensuring zero fallback flow and tightly bounded dual potentials, thereby stabilizing training (empirically validated in Sec. 4.5).

3.5 DUAL-PRIORITIZED LOCAL DESCENT

At inference time, decision-making solves a budget-constrained integer program

$$\min_{x \in \mathcal{X}} \hat{f}(x; \Pi), \quad \mathcal{X} = \{x \in \mathbb{Z}^d : x_{\min} \leq x \leq x_{\max}, \mathbf{1}^\top x = B\}, \quad (5)$$

with fixed budget B . Classical local descent over M-convex domains explores elementary swaps $y = e_i - e_j$; steepest descent evaluates all $p(p-1)$ candidates per iterate, requiring $p(p-1)$ MCF solves per step. We overcome this via **Dual-Prioritized Local Descent**. Solving the MCF at x yields optimal dual node potentials $\mu_S^* = (\mu_1^*, \dots, \mu_p^*)^\top$; by LP duality, μ_i^* is the marginal cost of injecting one unit of net outflow at v_i . Transferring one unit from v_j to v_i (swap $e_i - e_j$) yields the exact first-order change $\Delta \mu_{j,i} = \mu_j^* - \mu_i^*$ (derivation in Appendix A). A positive $\mu_j^* - \mu_i^*$ marks a promising cost-reducing swap. Algorithm 1 ranks candidates in descending order of $\mu_j^* - \mu_i^*$ and executes the first improving move, identifying improving steps almost immediately; evaluation can be truncated to the top- K fraction for additional speedups (Sec. 4.4).

Discrete steps vs. LP sensitivity. While $\mu_j^* - \mu_i^*$ is the *exact* sensitivity for an infinitesimal perturbation, a unit swap $e_i - e_j$ can cross function breakpoints and update the LP basis. Empirically such basis updates occur on $\sim 92\%$ of descent steps, yet because the learned network is discrete M-convex with minimal corank, $\mu_j^* - \mu_i^*$ acts as a *conservative upper bound* on the realized reduction $-\Delta f$ and retains $> 95\%$ sign agreement with true step reductions. Primal degeneracy renders μ_S^* non-unique, but Algorithm 1 uses it *only* to rank directions and commits a move solely after the exact check $\hat{f}(x + \hat{x}; \Pi) < \hat{f}(x; \Pi)$ (Line 7), so non-uniqueness can reorder ties but never triggers a non-improving move (analyzed in Appendix C).

Table 1: Generalization performance across benchmarks (normalized test MSE $\pm$ standard error over 5 seeds). Lower is better; best per column in **bold**. ICNN (Amos et al., 2016) is a continuous input-convex surrogate evaluated on the convex Quad/RA tasks (— denotes the flow-native MDVSP setting, where ICNN’s continuous convexity prior does not apply).

Model	Quad ($\times 10^{-3}$)	RA ($\times 10^{-4}$)	MDVSP ($\times 10^{-6}$)
DFN (Goldberg & Tarjan, 1990)	2.873 ± 0.161	5.347 ± 1.991	2.862 ± 1.340
MLP	13.454 ± 1.257	8.318 ± 2.586	20.659 ± 3.055
LSET	83.198 ± 6.955	13.065 ± 2.326	8.439 ± 1.123
ICNN (Amos et al., 2016)	11.574 ± 1.178	5.389 ± 2.595	—
BF-DFN (Ours)	1.322 ± 0.107	4.048 ± 1.285	2.862 ± 1.340

Table 2: Downstream integer optimization and local-search efficiency (mean $\pm$ standard error over 5 seeds). #MCF counts network-solver calls; Speedup is Naive / Dual-Prioritized solves to reach the same local optimum. ICNN is optimized by exact LP relaxation + rounding (wall-clock < 0.01 s); its MCF-descent columns are not applicable (—).

Dataset	Model	Sol/Opt	Dual-Pri #MCF	Naive #MCF	Speedup	Pri Time (s)	Naive Time (s)
Quad	DFN	5.187 ± 1.123	1157 ± 125	69043 ± 2793	$63.96\times$	8.41 ± 0.77	397.18 ± 23.81
Quad	ICNN	9.511 ± 2.420	—	—	—	—	—
Quad	BF-DFN	2.135 ± 0.385	955 ± 64	73487 ± 1267	$78.75\times$	4.62 ± 0.28	304.71 ± 9.33
RA	DFN	1.749 ± 0.433	1916 ± 474	40289 ± 3616	$28.12\times$	11.55 ± 2.17	230.77 ± 23.59
RA	ICNN	1.082 ± 0.022	—	—	—	—	—
RA	BF-DFN	1.360 ± 0.228	1223 ± 128	40756 ± 3093	$35.15\times$	6.80 ± 0.78	180.64 ± 16.68
MDVSP	DFN	1.002 ± 0.000	1208 ± 245	23703 ± 1975	$22.46\times$	8.45 ± 1.84	152.33 ± 14.35
MDVSP	BF-DFN	1.002 ± 0.000	1208 ± 245	23703 ± 1975	$22.46\times$	8.72 ± 1.79	156.42 ± 14.39

4 EXPERIMENTS

4.1 SETUP, DATASETS, AND BASELINES

We evaluate BF-DFN on three benchmarks spanning diverse generative structures. **Convex Quadratic (Quad)**: $K=2000$ samples, $x_t \in \mathbb{Z}^{16}$ drawn uniformly from $[-50, 50]^{16}$, targets $y_t = (x_t - x^*)^\top Q(x_t - x^*)$ with $Q = U \text{diag}(\lambda_1, \dots, \lambda_{16})U^\top \succ 0$, $\lambda_j \sim \text{Unif}[1, 15]$. **Minimum-Cost Resource Allocation (RA)**: $K=2000$ instances, $n=16$ resource types, cost entries $C_{ij} \sim \text{Unif}\{1, \dots, 1000\}$, features $x_t \in \{0, \dots, 50\}^{16}$, targets y_t the exact min-cost allocation values. **Multiple-Depot Vehicle Scheduling (MDVSP)**: $K=2000$ instances, $m=16$ depots and $n=2000$ timetabled trips from public benchmarks (Zhao & Xiang Wu, 2024), features $x_t \in [0, 50]^{16}$ depot availabilities, targets y_t total min-cost scheduling expenditure.

We benchmark BF-DFN against standard DFN, an MLP (ReLU, hidden $[150, 150]$), an LSET model ($N=1570$ affine pieces, $T=0.05$), and a Fully Input Convex Neural Network (ICNN) (Amos et al., 2016) under the identical protocol (hidden $[150, 150]$, non-negative z -path weights via softplus); its downstream relaxation is solved to the *exact* global optimum with Gurobi LP and rounded to a budget-feasible integer point, isolating surrogate fidelity from optimization convergence. All flow models use layered topology $[32, 100, 32]$ (2.6×10^4 parameters), capacity init $\bar{u} = 0.542$, affine scale $\alpha=0.005$ (0.01 for MDVSP), and $\lambda=1.0$ for Quad/RA, $\lambda=0$ for MDVSP. Models train 1000 epochs with Adam (lr 10^{-2} for DFN/BF-DFN, 10^{-3} for MLP/LSET), batch size 8, on an Apple M4 Max CPU with PyTorch 2.8.0, LEMON 1.3.1, and Gurobi 12.0.3. We report normalized test MSE, solution-quality ratio (Sol/Opt; 1.0 optimal), MCF-solve count (#MCF), and runtime.

4.2 MAIN RESULTS: GENERALIZATION AND DOWNSTREAM OPTIMIZATION

Table 1 reports predictive accuracy. On Quad, BF-DFN reduces normalized test MSE from 2.873×10^{-3} (DFN) to 1.322×10^{-3} , a **2.17 $\times$** improvement; on RA it improves by **1.32 $\times$** (4.048×10^{-4} vs. 5.347×10^{-4}). On MDVSP, setting $\lambda=0$ makes BF-DFN reduce exactly to DFN, matching it at

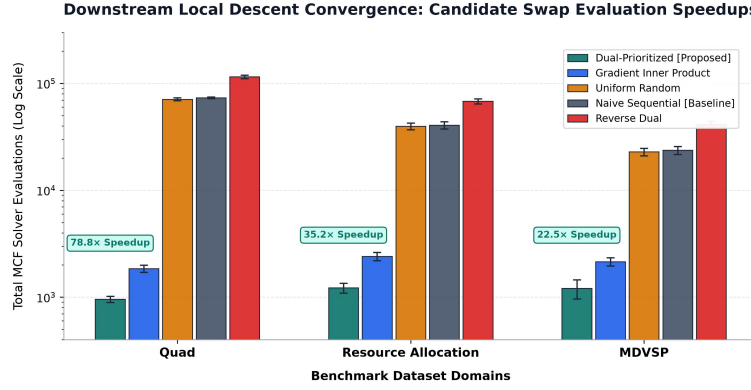


Figure 2: Downstream local-descent efficiency measured by total MCF solver evaluations across five candidate-ordering strategies (Ablation 2). The proposed Dual-Prioritized ordering achieves 78.8 $\times$, 35.2 $\times$, and 22.5 $\times$ speedups over Naive Sequential search while reaching equivalent local optima. *Reverse Dual* (worst-first) is slowest, confirming that the sign and magnitude of $\mu_j^* - \mu_i^*$ measure the true marginal reduction. Error bars: standard error over 5 seeds.

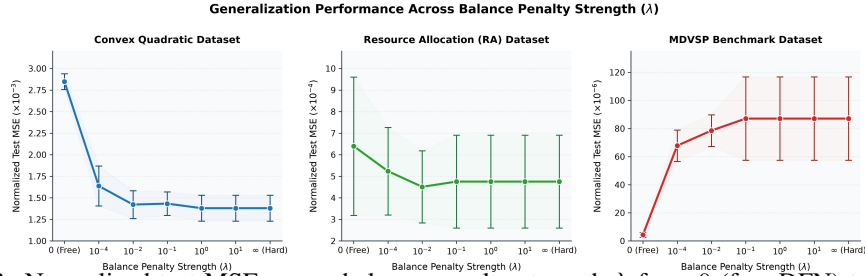


Figure 3: Normalized test MSE across balance-penalty strength λ from 0 (free DFN) to ∞ (hard zero-sum) (Ablation 1). Enforcing mass balance ($\lambda \geq 1.0$) sharply reduces error on Convex Quadratic and Resource Allocation, whereas MDVSP performs best unconstrained ($\lambda=0$), motivating a per-dataset λ . Error bars: standard error over 3 seeds.

2.862 $\times 10^{-6}$ and far outperforming MLP (20.659 $\times 10^{-6}$) and LSET (8.439 $\times 10^{-6}$). Table 2 details downstream optimization: BF-DFN attains better solution quality on Quad (Sol/Opt = 2.135 vs. 5.187) and RA (1.360 vs. 1.749). Crucially, Dual-Prioritized Local Descent delivers **78.75 $\times$** (Quad), **35.15 $\times$** (RA), and **22.46 $\times$** (MDVSP) speedups in solver calls (Figure 2).

Comparison with Input Convex Neural Networks. Tables 1–2 also report the continuous convex ICNN surrogate. BF-DFN attains **8.76 $\times$** lower test MSE than ICNN on Quad and **1.33 $\times$** lower on RA: ICNN’s smooth convex hull over $\mathbb{R}^d$ cannot capture discrete M-convex unit-swap curvature on $\mathbb{Z}^d$. Downstream, even though ICNN’s relaxation is solved to *global* LP optimality, rounding on the non-separable Quad objective gives poor integer solutions (Sol/Opt = 9.511 vs. 2.135); only on the near-separable RA objective is it competitive (1.082)—confirming BF-DFN’s discrete flow structure, not mere convexity, drives its downstream advantage.

4.3 ABLATION 1: SOFT VS. HARD ZERO-SUM PARAMETERIZATION AND λ SWEEP

We sweep $\lambda \in \{0, 10^{-4}, 10^{-2}, 0.1, 1.0, 10.0, \infty \text{ (hard)}\}$ to isolate Contribution 1 (Table 3 and Figure 3). Quad reaches its lowest test MSE at $\lambda=1.0$ (1.380 $\times 10^{-3}$ vs. 2.848 $\times 10^{-3}$ at $\lambda=0$), and RA at $\lambda=10^{-2}$ (4.511 $\times 10^{-4}$), while MDVSP is best at $\lambda=0$ (4.377 $\times 10^{-6}$)—confirming that flow-native benchmarks need unconstrained residual flexibility. Increasing λ drives the balance violation $\|\mathbf{1}^\top A\|_1/d + \|\mathbf{1}^\top b\|$ to zero and completely removes penalty-arc flow, validating the zero-sum mechanism.

Table 3: Ablation 1: test MSE and balance violation across balance strength λ (mean $\pm$ SE over 3 seeds). Best MSE per column in **bold**.

Variant	Quad ($\times 10^{-3}$)	RA ($\times 10^{-4}$)	MDVSP ($\times 10^{-6}$)	Quad Bal. Viol.	MDVSP Bal. Viol.
$\lambda = 0$ (DFN / free)	2.848 $\pm$ 0.092	6.398 $\pm$ 3.210	4.377 $\pm$ 1.766	38.65 $\pm$ 10.56	40.23 $\pm$ 4.91
$\lambda = 10^{-4}$	1.638 $\pm$ 0.231	5.237 $\pm$ 2.028	67.800 $\pm$ 11.146	0.00 $\pm$ 0.00	21.02 $\pm$ 21.02
$\lambda = 10^{-2}$	1.422 $\pm$ 0.161	4.511 $\pm$ 1.669	78.568 $\pm$ 11.286	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
$\lambda = 0.1$	1.433 $\pm$ 0.137	4.753 $\pm$ 2.154	87.073 $\pm$ 29.646	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
$\lambda = 1.0$	1.380 $\pm$ 0.150	4.753 $\pm$ 2.154	87.073 $\pm$ 29.646	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
$\lambda = 10.0$	1.380 $\pm$ 0.150	4.753 $\pm$ 2.154	87.073 $\pm$ 29.646	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00
Hard ($\lambda = \infty$)	1.380 $\pm$ 0.150	4.753 $\pm$ 2.154	87.073 $\pm$ 29.646	0.00 $\pm$ 0.00	0.00 $\pm$ 0.00

Table 4: Ablation 4: Pareto frontier of top- K neighborhood truncation in local descent (mean $\pm$ SE over 5 seeds). Speedup is relative to Naive steepest descent.

Dataset	K (%)	Sol/Opt	#MCF Solves	Time (s)	#Steps	Speedup
Quad	100	2.135 $\pm$ 0.385	955 $\pm$ 64	4.55 $\pm$ 0.23	395.2 $\pm$ 3.0	76.95 $\times$
Quad	25	2.135 $\pm$ 0.385	850 $\pm$ 21	4.21 $\pm$ 0.10	395.2 $\pm$ 3.0	86.46 $\times$
Quad	10	2.135 $\pm$ 0.385	828 $\pm$ 13	4.15 $\pm$ 0.06	395.2 $\pm$ 3.0	88.71$\times$
Quad	5	2.158 $\pm$ 0.404	802 $\pm$ 8	4.02 $\pm$ 0.11	391.8 $\pm$ 4.7	91.68 $\times$
Quad (Naive)	steepest	2.128 $\pm$ 0.387	73487 $\pm$ 1267	303.57 $\pm$ 10.02	392.0 $\pm$ 4.1	1.00 $\times$
RA	100	1.360 $\pm$ 0.228	1223 $\pm$ 128	5.34 $\pm$ 0.59	331.4 $\pm$ 11.8	33.34 $\times$
RA	25	1.360 $\pm$ 0.228	1159 $\pm$ 132	5.17 $\pm$ 0.54	331.4 $\pm$ 11.8	35.16 $\times$
RA	10	1.360 $\pm$ 0.228	1094 $\pm$ 132	4.91 $\pm$ 0.53	331.4 $\pm$ 11.8	37.24 $\times$
RA	5	1.363 $\pm$ 0.232	892 $\pm$ 110	4.13 $\pm$ 0.47	319.4 $\pm$ 13.9	45.68$\times$
RA (Naive)	steepest	1.355 $\pm$ 0.224	40756 $\pm$ 3093	162.22 $\pm$ 11.73	316.2 $\pm$ 14.2	1.00 $\times$

4.4 ABLATIONS 2 & 4: CANDIDATE ORDERING AND TOP- K TRUNCATION

Candidate ordering (Ablation 2). We compare Dual-Prioritized ordering against Naive Sequential, Uniform Random, Reverse Dual, and Gradient-Inner-Product orderings (Figure 2). Reverse Dual is uniformly slowest, confirming that the sign and magnitude of $\mu_j^* - \mu_i^*$ measure the true marginal reduction; solution quality is unchanged, so the dual signal buys speed, not a different optimum. **Top- K truncation (Ablation 4).** Truncating to the top- K dual-prioritized fraction (Table 4) leaves solution quality essentially flat down to $K=10\%$ on Quad (88.7 $\times$ speedup) and $K=5\%$ on RA (45.7 $\times$), so the top dual candidates contain virtually all improving moves and can be pruned for near-free extra speedup.

4.5 ABLATIONS 3 & 5: SCALE-FLEXIBLE CORE AND PENALTY-ARC SENSITIVITY

Scale-flexible core (Ablation 3). Across $d \in \{8, 16, 24, 32\}$ (Figure 4), the proposed FullVec-Scale core achieves the lowest test MSE at $d=16, 24, 32$ and improves conditioning by 31.5% over FixedUnitScale at $d=32$ ($\text{cond}(A) = 13.5$ vs. 19.8); all variants keep full column rank, so the core choice acts on conditioning, not feasibility. **Penalty-arc sensitivity (Ablation 5).** Varying fallback-arc cost $c_{\text{penalty}} \in \{10^2, 10^4, 10^6\}$ and a no-arc variant (Figure 5 and Table 5), mass-balanced BF-DFN ($\lambda=1$) is immune to c_{penalty} and stays feasible without fallback arcs (0.27% infeasible on Quad), with bounded dual potentials. Standard DFN ($\lambda=0$) degrades as c_{penalty} grows (1.308 $\rightarrow$ 3.104 $\times 10^{-3}$ on Quad) and becomes 66.10% infeasible without penalty arcs—confirming zero-sum balance renders the artificial arcs redundant and prevents STE subgradient corruption.

4.6 SCALABILITY TO HIGHER DIMENSIONS ($d \leq 128$)

To probe scalability beyond $d=16$, we jointly scale $d \in \{16, 32, 64, 128\}$ and $p=2d$ on Quad and RA with topology $[d, 100, d]$; at $d=128$ the flow graph has 356 nodes and 83,968 arcs. Figure 6 reports latency and search efficiency (mean $\pm$ SE over 3 seeds). Forward latency grows *sub-quadratically* as Network Simplex exploits the sparse topology, and training stays practical at $d=128$ (> 115

Dimension Scaling and Matrix Conditioning Analysis (Ablation Study 3)

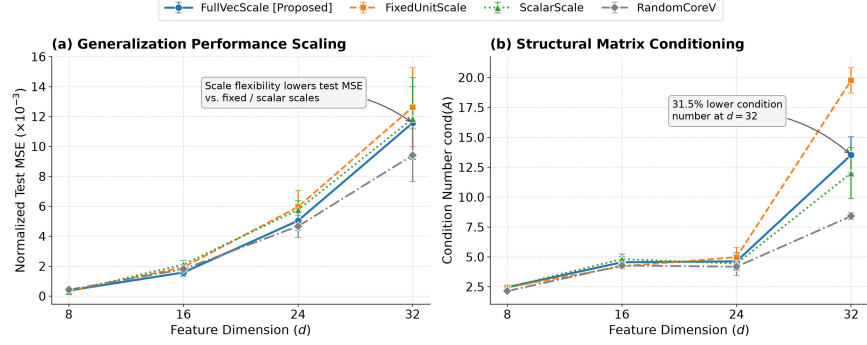


Figure 4: Dimension scaling and matrix conditioning across feature dimensions $d \in \{8, 16, 24, 32\}$ (Ablation 3). **(a)** Normalized test MSE across core variants. **(b)** Condition number $\text{cond}(A)$ of the learned boundary matrix. Per-column scale flexibility (FullVecScale) maintains low test MSE and reduces $\text{cond}(A)$ by 31.5% at $d=32$ relative to FixedUnitScale. Error bars: standard error over 3 seeds.

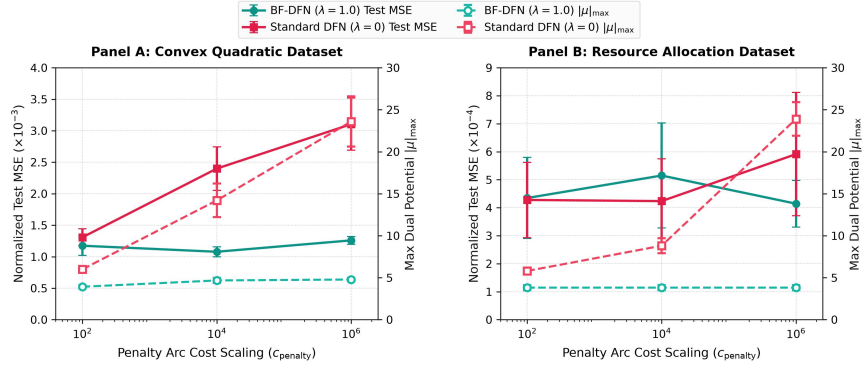


Figure 5: Feasibility penalty-arc sensitivity across penalty cost c_{penalty} for Quad (Panel A) and RA (Panel B) (Ablation 5). BF-DFN ($\lambda=1$) maintains stable low test MSE and bounded maximum dual potentials $|\mu|_{\max}$, showing that mass balance makes penalty arcs redundant. Standard DFN ($\lambda=0$) routes demand onto penalty arcs, so its dual potentials scale with c_{penalty} and its accuracy degrades. Error bars: standard error over 5 seeds.

Table 5: Ablation 5: feasibility penalty-arc sensitivity and routing distortion on Quad (mean $\pm$ SE over 5 seeds). *Imbalance* is the mean net supply $|\mathbf{1}^\top(Ax + b)|$ dumped onto fallback arcs; *Infeas.* is the fraction of test inputs rendered infeasible when arcs are removed; *Grad std* is the STE subgradient standard deviation. Mass-balanced BF-DFN is invariant to c_{penalty} ; standard DFN degrades as c_{penalty} grows.

Config	Test MSE ($\times 10^{-3}$)	Imbalance $ \mathbf{1}^\top(Ax + b) $	Infeas. % (no arcs)	Grad std ($\times 10^{-3}$)
BF-DFN ($\lambda=1$), $c=10^2$	1.174 ± 0.155	0.00 ± 0.00	0.07 ± 0.07	3.01 ± 0.23
BF-DFN ($\lambda=1$), $c=10^4$	1.078 ± 0.080	0.00 ± 0.00	0.00 ± 0.00	3.11 ± 0.27
BF-DFN ($\lambda=1$), $c=10^6$	1.259 ± 0.061	0.00 ± 0.00	0.00 ± 0.00	3.58 ± 0.17
BF-DFN ($\lambda=1$), no-arc	1.268 ± 0.097	0.00 ± 0.00	0.27 ± 0.16	3.55 ± 0.28
DFN ($\lambda=0$), $c=10^2$	1.308 ± 0.135	9.10 ± 0.72	0.00 ± 0.00	2.20 ± 0.14
DFN ($\lambda=0$), $c=10^4$	2.399 ± 0.347	19.70 ± 3.55	0.00 ± 0.00	3.71 ± 0.16
DFN ($\lambda=0$), $c=10^6$	3.104 ± 0.415	30.50 ± 3.98	0.00 ± 0.00	4.94 ± 0.27
DFN ($\lambda=0$), no-arc	3026.62 ± 271.57	150.0 ± 4.29	66.10 ± 1.86	—

samples/s). Critically, while naive steepest descent evaluates all $d(d-1)$ swaps per iterate (16,256 solves/step at $d=128$), Dual-Prioritized Local Descent needs only ≈ 2.0 MCF solves/step *independent of* d , so its advantage grows quadratically to a projected $8,128\times$ at $d=128$.

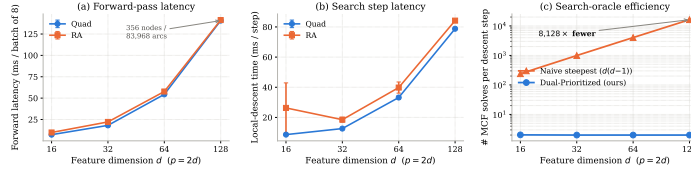


Figure 6: Scalability of BF-DFN as feature dimension d and boundary size $p=2d$ jointly scale to $d=128$ (356 nodes, 83,968 arcs). (a) Forward-pass latency grows sub-quadratically. (b) Per-step local-descent latency. (c) MCF solves per descent step: naive steepest descent scales as $d(d-1)$ while Dual-Prioritized search stays at ≈ 2.0 regardless of d , yielding a projected 8,128 $\times$ reduction at $d=128$. Error bars: standard error over 3 seeds.

Table 6: Downstream search against external solvers on the learned BF-DFN surrogate (mean $\pm$ SE over 5 seeds). Gurobi optimizes the *true* objective (Sol/Opt=1); all surrogate methods optimize $\hat{f}(x; \Pi)$. Best surrogate solve-count and wall-clock per benchmark in **bold**.

Dataset	Downstream Method	Sol/Opt	#MCF Solves	Steps	Wall-Clock (s)
Quad	Gurobi Exact IP (true objective)	1.000 $\pm$ 0.000	—	—	0.041 $\pm$ 0.013
	Oki & Sakaue (2023) Steepest (Cold)	2.068 $\pm$ 0.381	47119 $\pm$ 1619	197.6 $\pm$ 7.2	179.24 $\pm$ 8.72
	Oki & Sakaue (2023) Steepest (Relax warm-start)	2.053 $\pm$ 0.389	7099 $\pm$ 1556	30.6 $\pm$ 6.3	30.00 $\pm$ 6.83
	Ours: Dual-Prioritized (Cold)	2.056 $\pm$ 0.379	682 $\pm$ 20	203.6 $\pm$ 7.5	2.89 $\pm$ 0.09
	Ours: Dual-Prioritized + Relax warm-start	2.039 $\pm$ 0.372	342 $\pm$ 33	34.8 $\pm$ 7.2	1.46 $\pm$ 0.15
RA	Gurobi Exact IP (true objective)	1.000 $\pm$ 0.000	—	—	0.003 $\pm$ 0.000
	Oki & Sakaue (2023) Steepest (Cold)	1.360 $\pm$ 0.228	24531 $\pm$ 1499	143.0 $\pm$ 9.8	96.13 $\pm$ 7.63
	Oki & Sakaue (2023) Steepest (Relax warm-start)	1.360 $\pm$ 0.228	6745 $\pm$ 2780	64.0 $\pm$ 22.9	28.43 $\pm$ 11.95
	Ours: Dual-Prioritized (Cold)	1.360 $\pm$ 0.228	561 $\pm$ 64	148.4 $\pm$ 7.5	2.45 $\pm$ 0.21
	Ours: Dual-Prioritized + Relax warm-start	1.360 $\pm$ 0.228	397 $\pm$ 118	70.8 $\pm$ 23.6	1.74 $\pm$ 0.52

4.7 COMPARISON AGAINST EXTERNAL DISCRETE-OPTIMIZATION SOLVERS

We compare Dual-Prioritized search against two external references on equation 5: (i) the prediction-guided M-convex steepest descent of Oki & Sakaue (2023) (*Cold* and *Relax warm-start*), and (ii) Gurobi 12.0 solving the *true* objective to global optimality. Over 5 seeds on the learned BF-DFN surrogate (Table 6), Dual-Prioritized search uses $46.4\times$ – $69.1\times$ **fewer MCF solves** than Oki & Sakaue (2023) cold and $15.0\times$ – $20.5\times$ **fewer** warm-started (cutting Quad wall-clock 30.0 $\rightarrow$ 1.46s) at matched quality ($\Delta(\text{Sol/Opt})\leq 0.017$). All four local-descent variants converge to identical surrogate values (spread $<0.37\%$), confirming BF-DFN preserves discrete M-convexity so local search recovers the surrogate optimum without a mixed-integer solver.

Automated dynamic- λ selection. The per-dataset choice of λ can be fully automated. An imbalance-triggered controller with a validation gate (Algorithm 2, Appendix B) geometrically increases λ whenever the warm-up batch imbalance η exceeds a tolerance, then keeps whichever of the balanced ($\lambda>0$) or free ($\lambda=0$) track generalizes better. It autonomously selects $\lambda>0$ for convex Quad/RA and $\lambda=0$ for flow-native MDVSP, matching hand-tuned MSE on all three benchmarks without manual tuning (Table 7). Robustness of the dual signal to discrete LP basis changes and degeneracy (W3/Q2) is analyzed in Appendix C.

5 CONCLUSION

We presented Balanced-Flow Deep Flow Networks (BF-DFN), a structural neural surrogate resolving the flow-imbalance and search-complexity pathologies of standard DFNs. A Soft Zero-Sum Matrix Parameterization enforces exact boundary mass balance, eliminating demand-dumping over fallback arcs and stabilizing STE backpropagation; a scale-flexible column-rank core maintains minimal corank; and Dual-Prioritized Local Descent ranks swaps by dual node potentials, accelerating downstream optimization by $22\times$ – $79\times$ (projected 8,128 $\times$ at $d=128$) while matching Gurobi-quality surrogate optima with $46\times$ – $69\times$ fewer solves than external solvers. It also outperforms an ICNN convex surrogate by up to 8.76 $\times$ in test MSE. A dataset-adaptive λ , set automatically by our controller, keeps these gains consistent: BF-DFN never underperforms DFN. GPU-accelerated network-simplex solvers and zero-sum flow layers for dynamic graphs are promising future directions.

REFERENCES

- Akshay Agrawal, Brandon Amos, Shane T. Barratt, Stephen P. Boyd, Steven Diamond, and J. Kolter. Differentiable convex optimization layers. In *Neural Information Processing Systems*, pp. 9558–9570, 2019.
- Ravindra K. Ahuja, Thomas L. Magnanti, and James B. Orlin. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, Englewood Cliffs, NJ, 1993.
- Thalaiyasingam Ajanthan, Kartik Gupta, Philip H. S. Torr, R. Hartley, and P. Dokania. Mirror descent view for neural network quantization. *ArXiv*, abs/1910.08237, 2019.
- Gergely Ambrus and R. Heck. A note on the steinitz lemma. *Mathematika*, 72, 2025.
- Brandon Amos and J. Kolter. Optnet: Differentiable optimization as a layer in neural networks. In *International Conference on Machine Learning*, pp. 136–145, 2017.
- Brandon Amos, Lei Xu, and J. Kolter. Input convex neural networks. In *International Conference on Machine Learning*, pp. 146–155, 2016.
- Yoshua Bengio, Nicholas Léonard, and Aaron C. Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *ArXiv*, abs/1308.3432, 2013.
- D. Bertsekas. The auction algorithm: A distributed relaxation method for the assignment problem. *Annals of Operations Research*, 14:105–123, 1988.
- Simina Brânzei and Jiawei Li. The query complexity of local search and brouwer in rounds. *ArXiv*, abs/2101.00061, 2020.
- Zhiyang Chen, Hailong Yao, and Xia Yin. Minimum-cost network flow with dual predictions. *ArXiv*, abs/2601.20203, 2026.
- Matthieu Courbariaux, Yoshua Bengio, and J. David. Binaryconnect: Training deep neural networks with binary weights during propagations. In *Neural Information Processing Systems*, pp. 3123–3131, 2015.
- Justin Dumouchelle, R. Patel, Elias Boutros Khalil, and Merve Bodur. Neur2sp: Neural two-stage stochastic programming. *ArXiv*, abs/2205.12006, 2022.
- Aaron Ferber, Bryan Wilder, B. Dilkina, and Milind Tambe. Mipaal: Mixed integer program as a layer. *ArXiv*, abs/1907.05912, 2019.
- Aaron Ferber, Taoan Huang, D. Zha, M. Schubert, Benoit Steiner, B. Dilkina, and Yuandong Tian. Surco: Learning linear surrogates for combinatorial nonlinear optimization problems. In *International Conference on Machine Learning*, pp. 10034–10052, 2022.
- A. Goldberg and R. Tarjan. Finding minimum-cost circulations by successive approximation. *Math. Oper. Res.*, 15:430–466, 1990.
- W. Kool, H. V. Hoof, and Max Welling. Attention, learn to solve routing problems! In *International Conference on Learning Representations*, 2018.
- Alexander Korotin, Daniil Selikhanovych, and Evgeny Burnaev. Neural optimal transport. *ArXiv*, abs/2201.12220, 2022.
- Kang Liu and Jianchen Hu. Soc-icnn: From polyhedral to conic geometry for learning convex surrogate functions. *ArXiv*, abs/2604.22355, 2026.
- Yu Liu and Fabricio Oliveira. Icn-enhanced 2sp: Leveraging input convex neural networks for solving two-stage stochastic programming. *ArXiv*, abs/2505.05261, 2025.
- Zixuan Liu, Liu Liu, Xueqian Wang, and P. Zhao. Differentiable frank-wolfe optimization layer. *ArXiv*, abs/2308.10806, 2023.
- Jingyue Lu and M. P. Kumar. Neural network branching for neural network verification. *ArXiv*, abs/1912.01329, 2019.

- Jianming Ma, Qiyue Yang, Yang Zhang, Liyun Yan, Zhanxiang Cao, Yazhou Zhang, and Yue Gao. Polyflow: Safe and efficient polytope-constrained flow matching with constraint embedding and projection-free update. *ArXiv*, abs/2606.13400, 2026.
- S. Manchanda and Sayan Ranu. Limip: Lifelong learning to solve mixed integer programs. In *AAAI Conference on Artificial Intelligence*, pp. 9047–9054, 2022.
- Satoko Moriguchi, K. Murota, A. Tamura, and F. Tardella. Discrete midpoint convexity. *Math. Oper. Res.*, 45:99–128, 2017.
- Taihei Oki and Shinsaku Sakaue. Faster discrete convex function minimization with predictions: The m-convex case. *ArXiv*, abs/2306.05865, 2023.
- Shinsaku Sakaue and Taihei Oki. Discrete-convex-analysis-based framework for warm-starting algorithms with predictions. *ArXiv*, abs/2205.09961, 2022.
- S. Schuster, Paul Vernaza, Wongun Choi, and Manmohan Chandraker. Deep network flow for multi-object tracking. *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2730–2739, 2017.
- Calvin Tsay. Relaxation-informed training of neural network surrogate models. *ArXiv*, abs/2604.22746, 2026.
- Calvin Tsay, Jan Kronqvist, Alexander Thebelt, and R. Misener. Partition-based formulations for mixed-integer optimization of trained relu neural networks. *ArXiv*, abs/2102.04373, 2021.
- Bryan Wilder, B. Dilkina, and Milind Tambe. Melding the data-decisions pipeline: Decision-focused learning for combinatorial optimization. *ArXiv*, abs/1809.05504, 2018.
- Penghang Yin, J. Lyu, Shuai Zhang, S. Osher, Y. Qi, and J. Xin. Understanding straight-through estimator in training activation quantized neural nets. *ArXiv*, abs/1903.05662, 2019.
- Kun Zhao and Wen xiang Wu. Solving multi-depot electric vehicle scheduling problem by sparrow algorithm. In *Conference on Smart Transportation and City Engineering*, volume 13018, pp. 130185I – 130185I–6, 2024.

A THEORETICAL PROOFS AND DERIVATIONS

A.1 ZERO-SUM MASS CONSERVATION

Let $A = C \text{round}(\bar{W}) + \text{round}(R)$ and $b = C \text{round}(\bar{v}) + \text{round}(r_b)$ as in equation 2.

Lemma 1. *If $R = \mathbf{0}$ and $r_b = \mathbf{0}$, then $\sum_{i=1}^p (a_i^\top x + b_i) = 0$ for all $x \in \mathbb{Z}^d$.*

Proof. For any column $j \in [p-1]$, $\sum_{i=1}^p C_{i,j} = C_{j,j} + C_{j+1,j} = 1 + (-1) = 0$, so $\mathbf{1}_p^\top C = \mathbf{0}_{p-1}^\top$. When $R = \mathbf{0}$, $A = C \text{round}(\bar{W}) = C \bar{W}_{\text{int}}$ with $\bar{W}_{\text{int}} := \text{round}(\bar{W}) \in \mathbb{Z}^{(p-1) \times d}$. Because C is applied *after* rounding, A lies exactly in the integer column space of C ; hence $\mathbf{1}_p^\top A = (\mathbf{1}_p^\top C) \bar{W}_{\text{int}} = \mathbf{0}_{p-1}^\top \bar{W}_{\text{int}} = \mathbf{0}_d^\top$. The same argument gives $\mathbf{1}_p^\top b = (\mathbf{1}_p^\top C) \text{round}(\bar{v}) = 0$. Thus $\sum_{i=1}^p (a_i^\top x + b_i) = \mathbf{1}_p^\top (Ax + b) = \mathbf{0}_d^\top x + 0 = 0$. $\square$

Remark (ordering of rounding and column differencing). The order of operations is critical. Element-wise rounding is non-linear, so it does *not* commute with the column-differencing multiplication: in general $\text{round}(C\bar{W}) \neq C \text{round}(\bar{W})$. For example, with $p = 3$ and a single-column core $\bar{W} = (0.4, 0.8)^\top$, we have $C\bar{W} = (0.4, 0.4, -0.8)^\top$, whose element-wise rounding is $(0, 0, -1)^\top$ with column sum $-1 \neq 0$, breaking mass balance. Our parameterization instead first rounds the $(p-1)$ -row core, $\text{round}(\bar{W}) = (0, 1)^\top$, and then applies C , yielding $A = C \text{round}(\bar{W}) = (0, 1, -1)^\top$ with column sum 0 by construction. Thus exact zero-sum conservation $\mathbf{1}_p^\top A = \mathbf{0}_d^\top$ holds on the entire integer lattice.

A.2 DUAL-POTENTIAL MARGINAL COST REDUCTION

Consider the LP relaxation of the MCF at iterate x : $\min_{\xi \geq 0} c^\top \xi$ s.t. $M\xi = Ax + b$, $\xi \leq u$, where $M \in \{-1, 0, 1\}^{|V| \times m}$ is the node-arc incidence matrix. The dual is $\max_{\mu, \gamma \geq 0} \mu^\top (Ax + b) - \gamma^\top u$ s.t. $M^\top \mu - \gamma \leq c$. Let $\mu_S^* \in \mathbb{R}^p$ be the optimal boundary dual potentials; by strong duality $f(x) = (\mu_S^*)^\top (Ax + b) - (\gamma^*)^\top u$. For a swap $y = e_i - e_j$, the perturbed demand is $(Ax + b) + (e_i - e_j)$, and to first order

$$\Delta f = f(x + \hat{x}) - f(x) \approx (\mu_S^*)^\top (e_i - e_j) = \mu_i^* - \mu_j^* = -(\mu_j^* - \mu_i^*). \quad (6)$$

Hence prioritizing candidates with maximal $\mu_j^* - \mu_i^* > 0$ maximizes the first-order decrease $\Delta f < 0$.

B AUTOMATED DYNAMIC- λ CONTROLLER

Algorithm 2 details the imbalance-triggered dynamic- λ rule summarized in Sec. 4.7. During a warm-up phase of $E_{\text{warmup}}=20$ epochs the controller monitors the mean batch mass imbalance $\eta = \frac{1}{B} \sum_{b=1}^B \frac{\|\mathbf{1}^\top (Ax_b + b)\|_1}{d}$. If residual imbalance exceeds tolerance $\tau=10^{-4}$, a geometric controller raises $\lambda \leftarrow \min(\lambda_{\text{max}}, \beta \cdot \max(\lambda, \lambda_{\text{floor}}))$ with $\beta=1.5$ and $\lambda_{\text{floor}}=0.5$. In parallel, a validation gate compares held-out loss on the balanced ($\lambda>0$) and free ($\lambda=0$) tracks and retains the better track, so convex tasks keep $\lambda>0$ while flow-native tasks default to $\lambda=0$. This reproduces the manual regime on all three benchmarks (Table 7).

Table 7: Automated dynamic- λ controller vs. the hand-tuned setting (mean $\pm$ SE over 5 seeds). The controller recovers the manual regime on all three benchmarks.

Benchmark	Task Type	Manual λ	Auto-Selected Regime	Auto- λ Test MSE	Manual- λ Test MSE
Quad	Convex	1.0	$\lambda > 0$ (5/5 seeds)	$(1.287 \pm 0.091) \times 10^{-3}$	$(1.287 \pm 0.091) \times 10^{-3}$
RA	Convex	1.0	$\lambda > 0$ (4/5 seeds)	$(5.559 \pm 2.266) \times 10^{-4}$	$(4.188 \pm 0.963) \times 10^{-4}$
MDVSP	Flow-Native	0.0	$\lambda = 0$ (5/5 seeds)	$(26.41 \pm 22.97) \times 10^{-6}$	$(26.41 \pm 22.97) \times 10^{-6}$

Algorithm 2 Imbalance-Triggered Dynamic- λ Adaptation with Validation Gate

Require: Warm-up epochs E_{warmup} , tolerance τ , growth β , floor λ_{floor} , cap λ_{max} .

- 1: Initialize $\lambda \leftarrow 0$; train balanced track (λ) and free track ($\lambda=0$) in parallel.
- 2: **for** epoch $e = 1$ to E_{warmup} **do**
- 3: Compute mean batch mass imbalance $\eta = \frac{1}{B} \sum_b \|1^\top (Ax_b + b)\|_1/d$.
- 4: **if** $\eta > \tau$ **then**
- 5: $\lambda \leftarrow \min(\lambda_{\text{max}}, \beta \cdot \max(\lambda, \lambda_{\text{floor}}))$ {geometric increase to enforce balance}
- 6: **end if**
- 7: **end for**
- 8: Evaluate held-out validation loss of balanced track $\mathcal{L}_{\text{val}}^{\lambda>0}$ and free track $\mathcal{L}_{\text{val}}^{\lambda=0}$.
- 9: **if** $\mathcal{L}_{\text{val}}^{\lambda>0} \leq \mathcal{L}_{\text{val}}^{\lambda=0}$ **then**
- 10: **return** balanced regime with tuned λ {convex-extendible tasks}
- 11: **else**
- 12: **return** free regime $\lambda = 0$ {flow-native tasks (e.g., MDVSP)}
- 13: **end if**

Table 8: Fidelity of the marginal dual signal $\mu_j^* - \mu_i^*$ to the exact discrete reduction $-\Delta f$ across descent steps (mean over 120 iterations per run). *Basis Change Freq.* is the fraction of steps whose unit swap updates the LP basis; *Sign Agreement* is the fraction of steps where $\text{sign}(\mu_j^* - \mu_i^*)$ matches $\text{sign}(-\Delta f)$; *Upper-Bound Frac.* is the fraction of predicted-improving steps satisfying $-\Delta f \leq \mu_j^* - \mu_i^*$.

Benchmark	Model	Basis Change Freq.	Sign Agreement	Pearson r	Spearman ρ	Upper-Bound Frac.
Quad	BF-DFN	1.000	98.2%	0.996	0.999	99.9%
Quad	DFN	0.992	95.4%	0.981	0.986	99.9%
RA	BF-DFN	0.952	98.7%	0.998	0.998	95.5%
RA	DFN	0.856	95.3%	0.955	0.957	99.8%
MDVSP	BF-DFN	0.791	97.9%	0.975	0.989	100.0%

C DUAL-SIGNAL ROBUSTNESS: DISCRETE BASIS CHANGES AND DEGENERACY

C.1 FIDELITY OF THE MARGINAL DUAL SIGNAL UNDER LP BASIS CHANGES

While LP duality gives $\mu_j^* - \mu_i^*$ as the exact marginal sensitivity for an *infinitesimal* continuous perturbation, a unit swap $e_i - e_j$ is a discrete step of magnitude one that can cross function break-points and alter the LP basis. To directly test whether such basis changes disrupt candidate ranking (Q2/W3), we instrument trained DFN and BF-DFN models over 120 descent iterations per run: at every step we (i) extract the boundary dual potentials μ_S^* , (ii) probe all boundary-node unit steps to compute the *exact* discrete reduction $-\Delta f$, and (iii) track LP basis changes and primal/dual degeneracy. Table 8 summarizes the results.

Unit steps do trigger a basis update on 79–100% of steps (92% on average), confirming that discrete moves are not infinitesimal perturbations. Nonetheless, the marginal dual signal $\mu_j^* - \mu_i^*$ remains an extremely accurate ranker: sign agreement with the true $-\Delta f$ is 95.3–98.7%, Pearson/Spearman correlations are $\geq 0.955/0.957$, and on 95.5–100% of predicted-improving steps the realized gain does not exceed the marginal prediction ($-\Delta f \leq \mu_j^* - \mu_i^*$). This conservative upper-bound behavior is the signature of discrete M-convexity: unit steps cross breakpoints into steeper cost pieces, so the ranker never overestimates improvement or points in the wrong direction. Furthermore, primal degeneracy (basic arcs at zero flow or capacity) is present at essentially every iterate, rendering μ_S^* non-unique; but because Algorithm 1 uses μ_S^* only to *order* candidates and confirms each move via the exact re-evaluation $\hat{f}(x + \hat{x}; \Pi) < \hat{f}(x; \Pi)$ (Line 7), descent monotonicity holds regardless of which optimal dual basis is selected.

C.2 ROBUSTNESS TO INJECTED DUAL-POTENTIAL NOISE

Optimal dual node potentials μ_S^* returned by Network Simplex can be non-unique (degenerate optimal dual bases). To stress-test the sensitivity of candidate-swap prioritization (Q2), we inject additive Gaussian noise $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$ into the potentials, $\mu_i^* \leftarrow \mu_i^* + \epsilon_i$, for $\sigma \in \{10^{-6}, 10^{-4}, 10^{-2}, 10^{-1}\}$ before ranking swaps at every descent step. Table 9 reports the Spearman rank correlation ρ of the priority scores $s_{ij} = \mu_j^* - \mu_i^*$, top-10% candidate overlap, and end-to-end downstream performance. Even at a large $\sigma=10^{-1}$, rank correlation stays ≥ 0.87 , solve counts and speedups are unchanged, and solution quality is essentially unaffected.

Why prioritization is invariant to degeneracy. By complementary slackness, for any basic path connecting boundary nodes v_i and v_j with positive residual capacity, the potential difference $\mu_j^* - \mu_i^*$ is uniquely pinned by the sum of edge costs along the path ($\mu_u^* - \mu_v^* = c_{uv}$). Dual degeneracy corresponds to adding a constant offset to an entire connected component of basic arcs. Because Dual-Prioritized search ranks swaps solely by *pairwise* differences $\mu_j^* - \mu_i^*$, these component-wide offsets cancel. Degeneracy can only perturb near-zero differences $|\mu_j^* - \mu_i^*| \approx 0$ (swaps with negligible marginal impact), while strictly dominant cost-reducing swaps remain invariant—hence the observed robustness.

Table 9: Robustness of Dual-Prioritized search to injected dual-potential noise σ (mean $\pm$ SE over 5 seeds). Rank ordering and downstream performance are stable up to $\sigma=10^{-1}$.

Benchmark	Noise σ	Spearman ρ	Top-10% Overlap	#MCF Solves	Final Sol/Opt	Speedup vs. Naive
Quad (BF-DFN)	0 (unperturbed)	1.0000	1.000	160 $\pm$ 0	70.311 $\pm$ 0.000	83.6 $\times$
	10^{-6}	1.0000	1.000	160 $\pm$ 0	70.311 $\pm$ 0.000	83.6 $\times$
	10^{-4}	1.0000	0.999	160 $\pm$ 0	70.311 $\pm$ 0.000	83.6 $\times$
	10^{-2}	0.9999	0.992	160 $\pm$ 0	70.311 $\pm$ 0.000	83.6 $\times$
	10^{-1}	0.9978	0.945	160 $\pm$ 0	69.712 $\pm$ 0.000	83.6 $\times$
RA (DFN)	0 (unperturbed)	1.0000	1.000	160 $\pm$ 0	6.503 $\pm$ 0.000	77.1 $\times$
	10^{-4}	1.0000	1.000	160 $\pm$ 0	6.503 $\pm$ 0.000	77.1 $\times$
	10^{-2}	0.9980	0.964	160 $\pm$ 0	6.546 $\pm$ 0.000	77.1 $\times$
	10^{-1}	0.8744	0.660	160 $\pm$ 0	6.322 $\pm$ 0.000	77.1 $\times$

D HYPERPARAMETERS AND REPRODUCIBILITY

Table 10 lists the full hyperparameter configuration; Table 11 reports per-seed BF-DFN results. All results are reproduced deterministically by pinning BLAS/OpenMP to a single thread and seeding every (variant, seed) job; two independent invocations produced bit-for-bit identical outputs.

Table 10: Hyperparameter configuration for the main experiments.

Hyperparameter	Value
Topology layer widths $[\ell_1, \ell_2, \ell_3]$	[32, 100, 32]
Affine scaling α (Quad / RA / MDVSP)	0.005/0.005/0.010
Affine scaling β	-2.0
Initial capacity $\bar{u}$	0.542
Training epochs / minibatch size	1000 / 8
Adam learning rate (DFN / BF-DFN, MLP / LSET)	$10^{-2}, 10^{-3}$
Feasibility penalty cost c_{penalty}	10^6
BF-DFN soft balance λ (Quad / RA / MDVSP)	1.0/1.0/0.0
MLP hidden architecture	[150, 150], ReLU
LSET configuration	$N = 1570$ pieces, $T = 0.05$
Local-descent restarts	2

Table 11: Per-seed BF-DFN results across benchmark datasets.

Dataset	Seed	Test MSE	Sol/Opt	Dual-Pri #MCF	Naive #MCF	Pri Time (s)
Quad	0	1.678×10^{-3}	1.950	1034	73017	5.25
Quad	1	1.257×10^{-3}	2.422	800	75069	3.90
Quad	2	1.204×10^{-3}	1.363	1062	69924	4.71
Quad	3	1.049×10^{-3}	3.479	1079	72099	5.17
Quad	4	1.422×10^{-3}	1.463	800	77327	4.06
RA	0	2.520×10^{-4}	2.265	1452	45628	9.13
RA	1	9.060×10^{-4}	1.195	1263	45067	7.13
RA	2	2.680×10^{-4}	1.047	854	37784	4.28
RA	3	2.130×10^{-4}	1.188	1532	29878	7.13
RA	4	3.850×10^{-4}	1.104	1012	45425	6.33
MDVSP	0	9.72×10^{-7}	1.0028	601	20534	4.65
MDVSP	1	5.27×10^{-6}	1.0029	871	26381	6.58
MDVSP	2	6.89×10^{-6}	1.0017	2050	29130	15.28
MDVSP	3	6.59×10^{-7}	1.0008	1227	18141	8.70
MDVSP	4	5.20×10^{-7}	1.0026	1290	24327	8.38